

Deep Networks

Normalization

Introduction to Deep Learning · Class 15

Outline

Where the Condition Number Comes From

Normalizing the Inputs

Depth: Initialization

Putting It Together

Where the Condition
Number Comes From

Repairing the problem instead of the iteration

Class 14 spent its whole length working around a large condition number. It never asked where that number came from.

Classes 13 and 14

change the *step*

momentum, schedules, Adam

accept κ and work around it

one lever: the update rule

Today

change the *surface* the step is taken on

rescale the inputs, the weights, the activations

make κ small in the first place

one lever: the parameterisation

The condition number is not a fact of nature. It is a consequence of the units we happen to have measured things in, and of how the network is scaled.

Notation added today

μ_i, σ_i^2 mean and variance of quantity i

$\tilde{x}_{ni}$ a normalized input

$\hat{a}_{ni}$ a normalized pre-activation

D fan-in of a layer

σ_w^2 variance of the initializer

γ_i, β_i learnable scale and shift

K mini-batch size in this class

M units in a layer

- γ and β are learned by gradient descent alongside the weights, exactly like any other parameter

The Hessian of a linear model is a data matrix

For $E(\mathbf{w}) = \sum_n (\mathbf{w}^\top \mathbf{x}_n - y_n)^2$ the curvature does not depend on $\mathbf{w}$ at all:

$$\mathbf{H} = 2\mathbf{X}^\top \mathbf{X}, \quad H_{ij} = 2 \sum_n x_{ni} x_{nj}.$$

- The diagonal H_{ii} is the second moment of feature i : a feature measured on a large scale has large curvature
- The off-diagonal H_{ij} is the correlation between features i and j
- So κ is a property of the **data**, before any learning happens

Two things can wreck it: features on different *scales*, and features that are *correlated*. Only the first is easy to fix.

The scale of a feature, squared

Proposition 1 (Feature scale and the condition number)

Let two uncorrelated, centred features have standard deviations $\sigma_1 \leq \sigma_2$. Then the least-squares Hessian $\mathbf{H} = 2\mathbf{X}^\top\mathbf{X}$ has $\kappa = (\sigma_2/\sigma_1)^2$, and after rescaling each feature to unit variance, $\kappa = 1$.

- **Uncorrelated and centred means $\sum_n x_{n1}x_{n2} \approx 0$, so $\mathbf{H}$ is diagonal.**
- Its entries are $2N\sigma_1^2$ and $2N\sigma_2^2$, which are then its eigenvalues.
- Their ratio is $(\sigma_2/\sigma_1)^2$; dividing each feature by its own σ makes both entries $2N$. □

Height in metres beside a platelet count in the hundreds of thousands gives $\kappa \sim 10^{10}$ before a weight is learned.

The scale of a feature, squared

Proposition 1 (Feature scale and the condition number)

Let two uncorrelated, centred features have standard deviations $\sigma_1 \leq \sigma_2$. Then the least-squares Hessian $\mathbf{H} = 2\mathbf{X}^\top\mathbf{X}$ has $\kappa = (\sigma_2/\sigma_1)^2$, and after rescaling each feature to unit variance, $\kappa = 1$.

- Uncorrelated and centred means $\sum_n x_{n1}x_{n2} \approx 0$, so $\mathbf{H}$ is diagonal.
- **Its entries are $2N\sigma_1^2$ and $2N\sigma_2^2$, which are then its eigenvalues.**
- Their ratio is $(\sigma_2/\sigma_1)^2$; dividing each feature by its own σ makes both entries $2N$. □

Height in metres beside a platelet count in the hundreds of thousands gives $\kappa \sim 10^{10}$ before a weight is learned.

The scale of a feature, squared

Proposition 1 (Feature scale and the condition number)

Let two uncorrelated, centred features have standard deviations $\sigma_1 \leq \sigma_2$. Then the least-squares Hessian $\mathbf{H} = 2\mathbf{X}^\top\mathbf{X}$ has $\kappa = (\sigma_2/\sigma_1)^2$, and after rescaling each feature to unit variance, $\kappa = 1$.

- Uncorrelated and centred means $\sum_n x_{n1}x_{n2} \approx 0$, so $\mathbf{H}$ is diagonal.
- Its entries are $2N\sigma_1^2$ and $2N\sigma_2^2$, which are then its eigenvalues.
- **Their ratio is $(\sigma_2/\sigma_1)^2$; dividing each feature by its own σ makes both entries $2N$.** □

Height in metres beside a platelet count in the hundreds of thousands gives $\kappa \sim 10^{10}$ before a weight is learned.

Normalizing the Inputs

Data normalization

$$\mu_i = \frac{1}{N} \sum_{n=1}^N x_{ni}$$

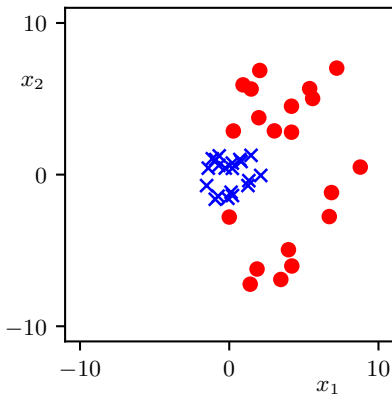
$$\sigma_i^2 = \frac{1}{N} \sum_{n=1}^N (x_{ni} - \mu_i)^2$$

$$\tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}$$

- Every input now has zero mean and unit variance across the training set
- Computed **once**, before training starts, because the inputs never change
- The **same** μ_i and σ_i must be applied to validation and test data

Bishop & Bishop (2024), Eqs. 7.48–7.50.

What normalizing the inputs does



- The red circles are the raw data for two variables on very different scales
- The blue crosses are the same points after each variable has been given zero mean and unit variance
- The rescaling is computed once, from the training set, and the same numbers are applied to validation and test data

Bishop & Bishop (2024), Fig. 7.7.

What normalizing the inputs does not fix

- It fixes the **diagonal** of $\mathbf{H}$; correlations live off the diagonal and survive untouched
- Removing those needs a rotation as well — whitening — which costs $O(d^3)$ and is rarely worth it
- A heavy-tailed feature can be normalized and still be dominated by a handful of points, so a transformation such as **log** often comes first
- Most importantly, it only touches the **first** layer

Layer two sees the outputs of layer one, not the data. Nothing so far controls what those look like.

Depth: Initialization

What one layer does to the variance

Proposition 2 (Variance across one ReLU layer)

Let $\mathbf{a}' = \mathbf{W}\mathbf{z}$ with $\mathbf{z} = \text{ReLU}(\mathbf{a})$, the entries of $\mathbf{W}$ drawn independently from $\mathcal{N}(0, \sigma_w^2)$, the biases zero, and $\mathbf{a}$ symmetric about zero with variance σ_a^2 . Then $\mathbb{E}[a'_i] = 0$ and

$$\sigma_{a'}^2 = \frac{1}{2} D \sigma_w^2 \sigma_a^2,$$

where D is the fan-in of the layer.

- $\mathbb{E}[a'_i] = \sum_j \mathbb{E}[w_{ij}] \mathbb{E}[z_j] = 0$, since $\mathbf{W}$ and $\mathbf{z}$ are independent and $\mathbf{W}$ has zero mean.
- So $\sigma_{a'}^2 = \mathbb{E}[a_i'^2] = \sigma_w^2 \sum_j \mathbb{E}[z_j^2]$.
- ReLU zeroes half of a symmetric distribution, giving $\mathbb{E}[z_j^2] = \frac{1}{2} \sigma_a^2$.
- Summing over the D inputs gives the result. □

Prince (2023), Eqs. 7.28–7.30; Bishop & Bishop (2024), Eqs. 7.21–7.22.

What one layer does to the variance

Proposition 2 (Variance across one ReLU layer)

Let $\mathbf{a}' = \mathbf{W}\mathbf{z}$ with $\mathbf{z} = \text{ReLU}(\mathbf{a})$, the entries of $\mathbf{W}$ drawn independently from $\mathcal{N}(0, \sigma_w^2)$, the biases zero, and $\mathbf{a}$ symmetric about zero with variance σ_a^2 . Then $\mathbb{E}[a'_i] = 0$ and

$$\sigma_{a'}^2 = \frac{1}{2} D \sigma_w^2 \sigma_a^2,$$

where D is the fan-in of the layer.

- $\mathbb{E}[a'_i] = \sum_j \mathbb{E}[w_{ij}] \mathbb{E}[z_j] = 0$, since $\mathbf{W}$ and $\mathbf{z}$ are independent and $\mathbf{W}$ has zero mean.
- **So $\sigma_{a'}^2 = \mathbb{E}[a_i'^2] = \sigma_w^2 \sum_j \mathbb{E}[z_j^2]$.**
- ReLU zeroes half of a symmetric distribution, giving $\mathbb{E}[z_j^2] = \frac{1}{2} \sigma_a^2$.
- Summing over the D inputs gives the result. □

Prince (2023), Eqs. 7.28–7.30; Bishop & Bishop (2024), Eqs. 7.21–7.22.

What one layer does to the variance

Proposition 2 (Variance across one ReLU layer)

Let $\mathbf{a}' = \mathbf{W}\mathbf{z}$ with $\mathbf{z} = \text{ReLU}(\mathbf{a})$, the entries of $\mathbf{W}$ drawn independently from $\mathcal{N}(0, \sigma_w^2)$, the biases zero, and $\mathbf{a}$ symmetric about zero with variance σ_a^2 . Then $\mathbb{E}[\mathbf{a}'] = 0$ and

$$\sigma_{a'}^2 = \frac{1}{2} D \sigma_w^2 \sigma_a^2,$$

where D is the fan-in of the layer.

- $\mathbb{E}[a'_i] = \sum_j \mathbb{E}[w_{ij}] \mathbb{E}[z_j] = 0$, since $\mathbf{W}$ and $\mathbf{z}$ are independent and $\mathbf{W}$ has zero mean.
- So $\sigma_{a'}^2 = \mathbb{E}[a_i'^2] = \sigma_w^2 \sum_j \mathbb{E}[z_j^2]$.
- **ReLU zeroes half of a symmetric distribution, giving $\mathbb{E}[z_j^2] = \frac{1}{2}\sigma_a^2$.**
- Summing over the D inputs gives the result. □

Prince (2023), Eqs. 7.28–7.30; Bishop & Bishop (2024), Eqs. 7.21–7.22.

What one layer does to the variance

Proposition 2 (Variance across one ReLU layer)

Let $\mathbf{a}' = \mathbf{W}\mathbf{z}$ with $\mathbf{z} = \text{ReLU}(\mathbf{a})$, the entries of $\mathbf{W}$ drawn independently from $\mathcal{N}(0, \sigma_w^2)$, the biases zero, and $\mathbf{a}$ symmetric about zero with variance σ_a^2 . Then $\mathbb{E}[a'_i] = 0$ and

$$\sigma_{a'}^2 = \frac{1}{2} D \sigma_w^2 \sigma_a^2,$$

where D is the fan-in of the layer.

- $\mathbb{E}[a'_i] = \sum_j \mathbb{E}[w_{ij}] \mathbb{E}[z_j] = 0$, since $\mathbf{W}$ and $\mathbf{z}$ are independent and $\mathbf{W}$ has zero mean.
- So $\sigma_{a'}^2 = \mathbb{E}[a_i'^2] = \sigma_w^2 \sum_j \mathbb{E}[z_j^2]$.
- ReLU zeroes half of a symmetric distribution, giving $\mathbb{E}[z_j^2] = \frac{1}{2} \sigma_a^2$.
- **Summing over the D inputs gives the result.**

□

Prince (2023), Eqs. 7.28–7.30; Bishop & Bishop (2024), Eqs. 7.21–7.22.

One layer is a multiplication, so depth is a power

Corollary 3 (Geometric growth or geometric decay)

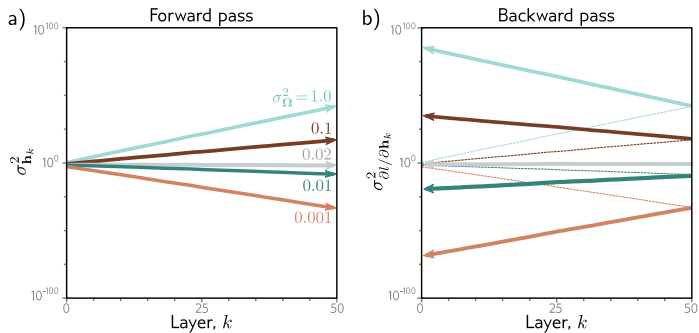
Applying Proposition 2 K times,

$$\sigma_{a^{(K)}}^2 = \left(\frac{1}{2}D\sigma_w^2\right)^K \sigma_{a^{(0)}}^2.$$

The variance therefore explodes if $\frac{1}{2}D\sigma_w^2 > 1$, vanishes if it is less than one, and is preserved only when it equals one exactly.

- There is no safe margin: any factor other than one, raised to the fiftieth power, is a catastrophe
- $D = 100$ with $\sigma_w^2 = 0.1$ gives a factor of 5, so fifty layers multiply the variance by $5^{50} \approx 10^{35}$

Fifty layers, five initializations



Fifty layers of one hundred units at five initialization variances. (a) the variance of the activations going forward; (b) the variance of the gradients coming back. Only $\sigma_w^2 = 2/D = 0.02$ stays level. Prince (2023), Fig. 7.7, where this variance is written σ_{Ω}^2 .

The same problem, seen through the chain rule

$$\frac{\partial E}{\partial w_i} = \sum_m \cdots \sum_l \sum_j \frac{\partial z_m^{(1)}}{\partial w_i} \cdots \frac{\partial z_j^{(K)}}{\partial z_l^{(K-1)}} \frac{\partial E}{\partial z_j^{(K)}}$$

- A gradient in the first layer is a product of K Jacobians: if most factors are below one it tends to zero, and above one to infinity
- These are the **vanishing** and **exploding** gradient problems, met in Class 11 as a property of the activation function and appearing here as a property of *scale*
- Matching the variance forward needs $\sigma_w^2 = 2/D$; backward it needs $2/D'$, with D' the fan-out

Bishop & Bishop (2024), Eq. 7.51; Prince (2023), Eq. 7.32.

He initialization

Proposition 4 (The variance that preserves scale)

Setting $\frac{1}{2}D\sigma_w^2 = 1$, that is

$$\sigma_w^2 = \frac{2}{D}, \quad \text{equivalently} \quad \sigma_w = \sqrt{\frac{2}{D}},$$

makes the variance of the pre-activations the same in every layer.

- The factor of two is exactly the half of the distribution that ReLU discards; without ReLU the rule is $1/D$
- Drawn from $\mathcal{N}(0, \sigma_w^2)$, or a uniform distribution with the same variance; the rule is per layer, so more inputs means smaller weights

He et al. (2015); Bishop & Bishop (2024), Eq. 7.23; Prince (2023), Eq. 7.31.

When the layer is not square

$$\underbrace{\sigma_w^2 = \frac{2}{D}}_{\text{forward}}$$

$$\underbrace{\sigma_w^2 = \frac{2}{D'}}_{\text{backward}}$$

$$\underbrace{\sigma_w^2 = \frac{4}{D + D'}}_{\text{the compromise}}$$

- If $D \neq D'$ the two conditions cannot both hold, so an average of the two is used
- This is **Glorot** initialization; it predates He and omits the factor of two, having been derived for sigmoidal units
- Biases start at zero, or at a small positive value for ReLU so that most units are active and passing gradient

Glorot & Bengio (2010); Prince (2023), Eq. 7.33.

Why the weights cannot all start equal

Proposition 5 (Symmetry is never broken by gradient descent)

Two hidden units that receive the same inputs and are initialized with identical incoming and outgoing weights compute the same function, receive the same gradient, and remain identical after every update.

- **Identical incoming weights and inputs give identical pre-activations, hence identical activations.**
- Identical outgoing weights give both the same $\partial E / \partial z_j$.
- Each weight gradient is that quantity times the same input, so the updates coincide, and by induction they stay identical. □

Bishop & Bishop (2024), §7.2.5.

Why the weights cannot all start equal

Proposition 5 (Symmetry is never broken by gradient descent)

Two hidden units that receive the same inputs and are initialized with identical incoming and outgoing weights compute the same function, receive the same gradient, and remain identical after every update.

- Identical incoming weights and inputs give identical pre-activations, hence identical activations.
- **Identical outgoing weights give both the same $\partial E / \partial z_j$.**
- Each weight gradient is that quantity times the same input, so the updates coincide, and by induction they stay identical. □

Bishop & Bishop (2024), §7.2.5.

Why the weights cannot all start equal

Proposition 5 (Symmetry is never broken by gradient descent)

Two hidden units that receive the same inputs and are initialized with identical incoming and outgoing weights compute the same function, receive the same gradient, and remain identical after every update.

- Identical incoming weights and inputs give identical pre-activations, hence identical activations.
- Identical outgoing weights give both the same $\partial E / \partial z_j$.
- Each weight gradient is that quantity times the same input, so the updates coincide, and by induction they stay identical. $\square$

Bishop & Bishop (2024), §7.2.5.

Initialization is a statement about the first step

The weights are chosen so that every layer starts at unit variance. Then training changes the weights.

- Nothing in the argument of the last section survives the first update, because it was an argument about a particular distribution of weights
- After a few hundred steps the deeper layers are nowhere near where they were placed
- The deeper the network, the sooner and the worse the drift
- The same reasoning that motivated normalizing the inputs applies to *every* layer — with one difference: it must be redone continuously

Bishop & Bishop (2024), §7.4.2.

Batch normalization

$$\mu_i = \frac{1}{K} \sum_{n=1}^K a_{ni}$$

$$\sigma_i^2 = \frac{1}{K} \sum_{n=1}^K (a_{ni} - \mu_i)^2$$

$$\hat{a}_{ni} = \frac{a_{ni} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}$$

- The sums run over the K members of the **mini-batch**, separately for each hidden unit i
- Exactly the input-normalization formula, applied to a hidden layer and recomputed at every step
- Either the pre-activations or the activations may be normalized

Ioffe & Szegedy (2015); Bishop & Bishop (2024), Eqs. 7.52–7.54.

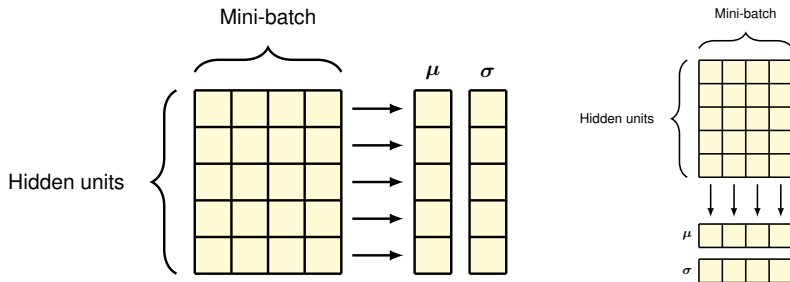
Giving the scale back, on purpose

$$\tilde{a}_{ni} = \gamma_i \hat{a}_{ni} + \beta_i$$

- Forcing zero mean and unit variance removes degrees of freedom, so two learnable parameters per unit hand them back
- It looks as though this undoes the normalization: the mean and variance can be anything again
- The difference is **how they are reached**: before, a complicated function of every weight below; now, two parameters set directly

Bishop & Bishop (2024), Eq. 7.55.

Which entries share a mean



Left: in batch normalization the mean and variance are computed across the mini-batch, separately for each hidden unit. Right: in layer normalization they are computed across the hidden units, separately for each data point. Bishop & Bishop (2024), Fig. 7.8.

Training time and inference time differ

$$\bar{\mu}_i^{(\tau)} = \alpha \bar{\mu}_i^{(\tau-1)} + (1 - \alpha) \mu_i, \quad \bar{\sigma}_i^{(\tau)} = \alpha \bar{\sigma}_i^{(\tau-1)} + (1 - \alpha) \sigma_i$$

- At prediction time there is no mini-batch: a single example has no variance of its own
- Evaluating them over the training set afterwards would work, and is too expensive, so running averages are accumulated *during* training and used at inference
- The network therefore computes a slightly different function in the two phases

B&B (2024), Eqs. 7.56–7.57

A normalized layer does not care how big its weights are

Proposition 6 (Scale invariance)

Let a layer's pre-activations be normalized. Multiplying every weight feeding into it by $a > 0$ leaves the layer's output unchanged, and scales the gradient with respect to those weights by $1/a$.

- **Scaling W by a scales the pre-activations, their mean and their standard deviation all by a .**
- The ratio $(a - \mu)/\sigma$ is therefore unchanged, so $E(aW) = E(W)$ for every a .
- Differentiating in W gives $a \nabla E(aW) = \nabla E(W)$. □

A layer whose weights have grown takes smaller steps: a per-layer learning rate nobody had to tune.

A normalized layer does not care how big its weights are

Proposition 6 (Scale invariance)

Let a layer's pre-activations be normalized. Multiplying every weight feeding into it by $a > 0$ leaves the layer's output unchanged, and scales the gradient with respect to those weights by $1/a$.

- Scaling W by a scales the pre-activations, their mean and their standard deviation all by a .
- **The ratio $(a - \mu)/\sigma$ is therefore unchanged, so $E(aW) = E(W)$ for every a .**
- Differentiating in W gives $a \nabla E(aW) = \nabla E(W)$. □

A layer whose weights have grown takes smaller steps: a per-layer learning rate nobody had to tune.

A normalized layer does not care how big its weights are

Proposition 6 (Scale invariance)

Let a layer's pre-activations be normalized. Multiplying every weight feeding into it by $a > 0$ leaves the layer's output unchanged, and scales the gradient with respect to those weights by $1/a$.

- Scaling W by a scales the pre-activations, their mean and their standard deviation all by a .
- The ratio $(a - \mu)/\sigma$ is therefore unchanged, so $E(aW) = E(W)$ for every a .
- **Differentiating in W gives $a \nabla E(aW) = \nabla E(W)$.** □

A layer whose weights have grown takes smaller steps: a per-layer learning rate nobody had to tune.

Why it works is not what was first claimed

- The original argument was **internal covariate shift**: updating early layers changes the distribution that later layers see, and normalizing removes that
- Later work found covariate shift is not the significant factor. Deliberately injecting distribution shift after a normalization layer does not undo the benefit
- What does change is the **smoothness** of the error surface: with normalization, the error and the gradient vary far less over the distance of one step
- A gradient that predicts the next gradient well is a gradient you can take a large step along — which is exactly what Class 14 was short of

Santurkar et al. (2018); Bishop & Bishop (2024), §7.4.2.

Three places batch normalization breaks

- **Small batches.** The statistics are estimates from K samples, with standard error $\propto 1/\sqrt{K}$; below a few dozen the correction is mostly noise
- **Distributed training.** The batch is split across devices, and pooling across them costs a synchronisation at every layer
- **Sequences.** In a recurrent network the distribution changes at every time step, so there is no fixed population

All three have the same cause: the statistics are taken across the *batch*. Take them across the *units* instead and all three disappear.

Bishop & Bishop (2024), §7.4.3.

Layer normalization

$$\mu_n = \frac{1}{M} \sum_{i=1}^M a_{ni}$$

$$\sigma_n^2 = \frac{1}{M} \sum_{i=1}^M (a_{ni} - \mu_n)^2$$

$$\hat{a}_{ni} = \frac{a_{ni} - \mu_n}{\sqrt{\sigma_n^2 + \varepsilon}}$$

- The identical expression with the sum over the M **units** instead of the K examples, and the same learnable γ_i, β_i
- Each example is normalized on its own, so training and inference compute the **same** function and no running averages are needed
- Introduced for recurrent networks; now standard in transformers

Ba, Kiros & Hinton (2016); Bishop & Bishop (2024), Eqs. 7.58–7.60.

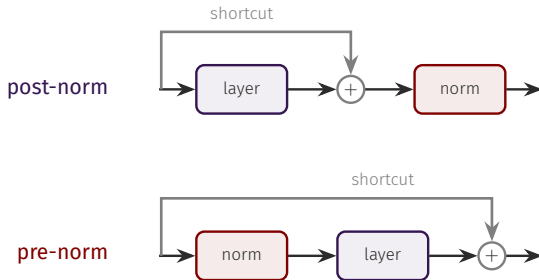
Four normalizers, one formula

Method	Average taken over	Used when
batch norm	the batch, per unit	convolutional vision, large batches
layer norm	the units, per example	transformers, recurrent networks
group norm	a block of units, per example	vision with small batches
instance norm	one unit, per example	style transfer, image generation

- All four subtract a mean, divide by a standard deviation, then apply a learnable γ and β ; they differ **only** in which entries are pooled
- Group norm interpolates: one group is layer norm, and as many groups as units is instance norm

Wu & He (2018).

Where the normalizer sits in a residual block

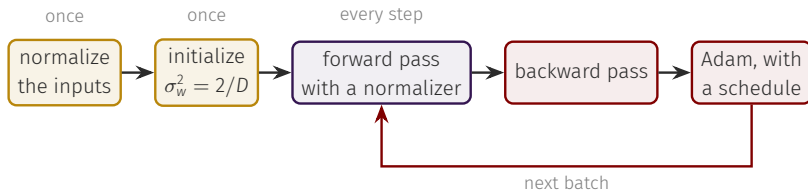


- Post-norm puts the normalizer on the trunk, so every shortcut is rescaled on the way back; pre-norm leaves the shortcut clean
- That is why deep stacks use pre-norm, and why post-norm needs a warm-up

Xiong et al. (2020)

Putting It Together

A network that trains



- The amber boxes are today's one-time work; the normalizer inside the forward pass is today's per-step work
- The maroon boxes are Classes 11, 13 and 14

What each layer of defence buys

Measure	Fixes	Defeated by
normalize the inputs	the scale of the first layer	depth
He or Glorot initialization	the scale of every layer, at $\tau = 0$	training
batch normalization	the scale of every layer, always	a small batch
layer normalization	the same, without using the batch	—

Each row is defeated by the next problem down, and the last row is where the sequence stops. That is the shape of the whole subject: every fix is a response to the failure of the one before it.

Takeaways

1. The condition number is set by the data and the parameterisation: for a linear model $\mathbf{H} = 2\mathbf{X}^T\mathbf{X}$
2. Rescaling each input fixes the diagonal of $\mathbf{H}$ exactly (Prop. 1), and nothing about correlations
3. One ReLU layer multiplies the variance by $\frac{1}{2}D\sigma_w^2$ (Prop. 2), and depth raises that factor to the K th power (Cor. 3)
4. $\sigma_w^2 = 2/D$ makes that factor one, and random values are what break the symmetry between units (Proposition 5)
5. Initialization constrains only the first step, so batch normalization redoes it at every step, with γ and β giving the scale back deliberately
6. It works by smoothing the error surface, not by removing covariate shift, and makes a layer scale-invariant (Prop. 6)
7. Layer normalization is the same formula averaged over the units, removing every dependence on K

References

- C. M. Bishop & H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. §7.2.5 and §7.4 (Eqs. 7.19–7.23, 7.48–7.60, Figs. 7.7–7.8).
- S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. §7.5 (Eqs. 7.26–7.33, Fig. 7.7).
- X. Glorot & Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks”, *AISTATS*, 2010.
- K. He, X. Zhang, S. Ren & J. Sun, “Delving deep into rectifiers”, *ICCV*, 2015.
- S. Ioffe & C. Szegedy, “Batch normalization: accelerating deep network training by reducing internal covariate shift”, *ICML*, 2015.
- J. L. Ba, J. R. Kiros & G. E. Hinton, “Layer normalization”, arXiv:1607.06450, 2016.
- Y. Wu & K. He, “Group normalization”, *ECCV*, 2018.
- S. Santurkar, D. Tsipras, A. Ilyas & A. Madry, “How does batch normalization help optimization?”, *NeurIPS*, 2018.
- R. Xiong et al., “On layer normalization in the transformer architecture”, *ICML*, 2020.

Questions?